

Programmazione 1

Corso c

14/10/2023
> Array e Matrici

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output scalare** con **input scalari** (es. `isOdd`).

Due possibilità:

1. La funzione chiamante passa l'input alla chiamata per **valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante attraverso il **return**
2. La funzione chiamante passa l'input alla chiamata **per valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante per **riferimento *indiretto/simulato* attraverso un parametro formale di tipo puntatore**

ATTENZIONE: nel caso 2 l'output deve essere per forza un puntatore!!!

Funzione Chiamante e Funzione Chiamata

- Perché l'input non viene passato per riferimento indiretto/simulato?

Golden Rule:

The usual rule is to pass quantities by value unless the program needs to alter the value, in which case you pass a pointer (CPrimerPlus)

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output vettoriale** con **input vettoriale** (es. `onlyOdd`).

Una sola possibilità:

1. La funzione chiamante passa il vettore in input alla chiamata per riferimento *simulato* attraverso 2 parametri formali (1p e 1int/size_t) e la chiamata passa l'output alla chiamante per riferimento *simulato* attraverso 2 parametri formali (1p e 1int/size_t)

Dove sono allocati i vettori? Perché?

Funzione Chiamante e Funzione Chiamata

- Un INPUT (vettoriale) può coincidere con l'OUTPUT (vettoriale)?
- Esempio:
 - `scanf` (scalare)
 - `cubo(vettoriale)`
- Non voglio permettere di variare il parametro: `const`

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Filtri 1: IN vettore, OUT vettore

- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a il cui valore è dispari;
- `void filtroPosizioniPariIt(int a[], size_t aLen, int b[], size_t bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano posizioni pari;
- `void filtroPosizioniDispariElementiPariIt(int a[], size_t aLen, int b[], size_t bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano una posizione dispari avendo, simultaneamente, valore pari;

Filtri 2: IN vettore, OUT vettore

- `void filtroIntervalloIt (int a[], size_t aLen, int b[], size_t bLen, int min, int max)` **modifica b**, memorizzandovi tutti e soli gli elementi di a il cui valore è compreso nell'intervallo [min, max], in cui gli estremi sono inclusi;
- `void filtroIndiciDoppioRiferimento (int a[], size_t aLen, int b[], size_t bLen, int riferimento)` **modifica b**, memorizzandovi tutti e soli gli indici degli elementi di a che hanno valore doppio di riferimento.

multipliTre: IN vettori, OUT vettore

Scrivere una funzione C iterativa di nome `multipliTre` con le seguenti caratteristiche:

- `multipliTre` dipende almeno dai seguenti parametri formali:
 - due array `a` e `b` di tipo intero, con dimensione massima `L_MAX > 0`, visti come input e tali che la parte effettivamente inizializzata sia delimitata dai valori in `aLen` e `bLen`, e un array `r` di tipo intero, con dimensione massima `L_MAX > 0`, visto come output;
- `multipliTre` restituisce `r` modificata come segue:
 - Per ogni indice `i` nell'intervallo `[0,minimo{aLen, bLen})`, in cui l'estremo inferiore è incluso e quello superiore escluso, l'elemento `r[i]` assume il valore `true` se almeno uno tra gli elementi di posizione `i` in `a` e `b` è multiplo di 3.
- `multipliTre` modifica anche il valore di `rLen` coerentemente con l'idea che un array, in questo caso `r`, è completamente descritto quando è noto il numero di elementi effettivamente utilizzati in esso.

IN array, OUT scalare

- Scrivere una funzione `C int sommaTuttiElementi (int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutti gli elementi in `a`.
- Scrivere una funzione `C int moltiplicaTuttiElementi (int a[], int la)` che, applicata ad un array `a`, restituisce la moltiplicazione di tutti gli elementi in `a`.
- Scrivere una funzione `C float sommaFrazioniTuttiElementi (int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutte le frazioni che si ottengono dividendo ogni elemento in `a` col successivo, ammesso che quest'ultimo esista.

CodeRunner

- Lab05-Es1 Somma selettiva
- Lab06-Es1 Statistiche sequenze